BiCut: A Solver-Agnostic Bilevel Optimization Compiler with Intersection Cuts for MIBLPs

BiCut Contributors

Abstract

We present BiCut, a typed bilevel optimization compiler that transforms mixed-integer bilevel linear programs (MIBLPs) into solver-ready single-level reformulations with machine-checkable correctness certificates. BiCut introduces *bilevel intersection cuts*—a new family of valid inequalities derived by extending Balas's (1971) intersection cut framework to the bilevel-infeasible set defined by follower suboptimality. For LP lower levels, we prove that the bilevel-infeasible set is a finite union of polyhedra, establish a polynomial-time separation oracle (for fixed follower dimension), and prove finite convergence of the cut loop under non-degeneracy. The compiler supports four reformulation strategies (KKT, strong duality, value function, column-and-constraint generation), three complementarity encodings, and emits to Gurobi, SCIP, and HiGHS. On our current generated suite of 122 bilevel instances, the KKT reformulation solved by HiGHS performs well (121/122 instances solved, most at the root node). The intersection cut framework is theoretically grounded, but in this preliminary study the cuts do not yet improve upon HiGHS's native cut machinery on these instances. BiCut is the first bilevel compiler with correctness certificates and the first tool providing bilevel-specific cutting planes.

1 Introduction

Bilevel optimization—where one optimization problem is nested as a constraint within another—arises in network interdiction [12], competitive facility location [13], strategic bidding in electricity markets [17], Stackelberg security games [20], and adversarial robustness certification [18]. The canonical mixed-integer bilevel linear program (MIBLP) has the form:

$$\begin{aligned} \min_{x \in X} \quad & d^\top x + e^\top y \\ \text{s.t.} \quad & Cx + Dy \le h, \\ & y \in S(x) = \operatorname*{argmin}_{y \ge 0} \{c^\top y : Ay \le b + Bx\}, \end{aligned} \tag{1}$$

where the *leader* controls x and the *follower* responds optimally with $y = S(x)$.

Despite two decades of algorithmic progress on MIBLPs [6, 4], the software landscape remains fragmented. Existing tools—BilevelJuMP.jl [8], PAO [11], GAMS EMP, YALMIP [14]—support one or two reformulation strategies with no automatic selection, no correctness guarantees, and no principled handling of mixed-integer lower levels.

This paper makes two contributions:

1. **Bilevel intersection cuts.** We extend Balas's 1971 intersection cut framework [1] to the bilevel-infeasible set, providing: (i) a polyhedrality theorem for LP lower levels (Theorem 2), (ii) a polynomial-time separation oracle for fixed follower dimension (Proposition 3), and (iii) a finite convergence result (Theorem 4).

2. **A bilevel optimization compiler.** BiCut provides a typed IR with leader/follower scoping, automatic reformulation selection among four strategies, machine-checkable correctness certificates, and solver-agnostic emission to Gurobi, SCIP, and HiGHS.

1.1 Related Work

Bilevel algorithms. The branch-and-cut approach of Fischetti et al. [9] solves MIBLPs by branching on the bilevel-infeasible set and generating intersection cuts in the (x,y)-space, but without the systematic framework we develop here. MibS [19] implements a specialized branch-and-bound for MIBLPs. DeNegre [7] studied bilevel integer programming via branch-and-cut. Xu and Wang [22] propose an exact algorithm under simplifying assumptions. None of these provide a compiler abstraction, correctness certificates, or solver-agnostic emission.

Bilevel software. BilevelJuMP.jl supports KKT and strong duality reformulations for linear bilevel programs [8]. PAO provides Python-based bilevel modeling within Pyomo [11]. Neither tool offers automatic reformulation selection, correctness certificates, or bilevel-specific cutting planes.

Intersection cuts. Balas [1] introduced intersection cuts for integer programming. Subsequent work by Balas and Margot [2], Conforti et al. [5], and Basu et al. [3] extended the framework to split cuts, multi-row cuts, and general lattice-free sets. Our work is the first to apply intersection cuts to bilevel feasibility.

KKT-based reformulation. The KKT reformulation replaces follower optimality with stationarity and complementarity conditions [6, 10]. Pineda and Morales [16] provide automatic big-M computation via bound-tightening LPs. CPLEX and Gurobi provide limited native bilevel support through indicator constraints, but without systematic reformulation selection.

Penalty-based methods. Penalty-based approaches relax the bilevel structure by penalizing follower suboptimality in the objective [21, 15]. These methods are simple to implement but provide only approximate solutions with no finite-time optimality guarantee for general MIBLPs.

2 Bilevel Intersection Cuts

2.1 Preliminaries

Consider the MIBLP (1) with an LP lower level. Define the *bilevel-feasible set*:

$$S_{\mathrm{BF}} = \{(x, y) : Cx + Dy \leq h,\ Ay \leq b + Bx,\ y \geq 0,\ c^\top y \leq \varphi(x)\},$$

where $\varphi(x) = \min\{c^\top y : Ay \leq b+Bx,\ y \geq 0\}$ is the lower-level *value function*. The *bilevel-infeasible set* is:

$$\bar{B} = \{(x, y) : Cx + Dy \leq h,\ Ay \leq b + Bx,\ y \geq 0,\ c^\top y > \varphi(x)\}.$$

Definition 1 (Critical Region)**.** *For a basis $\mathcal{B}$ of the lower-level LP, the* critical region *is $C_{\mathcal{B}} = \{x : A_{\mathcal{B}}^{-1}(b + Bx) \geq 0,\ c^\top A_{\mathcal{B}}^{-1}(b + Bx) \leq c^\top A_{\mathcal{B}'}^{-1}(b + Bx)\ \forall \mathcal{B}'\}$, where $A_{\mathcal{B}}$ denotes the basis submatrix.*

Algorithm 1 Bilevel Intersection Cut Separation

Require: LP relaxation optimum $(\hat{x}, \hat{y})$, lower-level data (A, b, B, c)
Ensure: Valid intersection cut or "bilevel-feasible"

```
Compute φ(x̂) via LP solve
if c^⊤ ŷ ≤ φ(x̂) + ε then
    return "bilevel-feasible"
end if
Identify critical region C_k containing x̂
Compute simplex tableau rays {d_j}_{j=1}^n at (x̂, ŷ)
for each ray d_j do
    Compute α_j = max{α : (x̂, ŷ) + α d_j ∈ B̄_k}
    (Boundary intersection via LP feasibility check)
end for
return Intersection cut: Σ_j x_j/α_j ≥ 1
```

2.2 Polyhedrality of the Bilevel-Infeasible Set

Theorem 2 (Polyhedrality)**.** *For an MIBLP with LP lower level where the lower-level feasible region $\{y : Ay \leq b + Bx,\ y \geq 0\}$ is bounded for all feasible x, the bilevel-infeasible set $\bar{B}$ is a finite union of (relatively open) polyhedra:*

$$\bar{B} = \bigcup_{k=1}^{K} \bar{B}_k,$$

where K is bounded by the number of bases of the lower-level LP, and each $\bar{B}_k$ is defined by a critical region and a strict suboptimality condition.

Proof sketch. The lower-level LP has finitely many bases $\mathcal{B}_1, \ldots, \mathcal{B}_K$. Each basis $\mathcal{B}_k$ defines a polyhedral critical region C_k in the leader's variable space. On C_k, the value function is affine: $\varphi(x) = c^\top A_{\mathcal{B}_k}^{-1}(b + Bx)$. A point (x, y) is bilevel-infeasible if and only if $x \in C_k$ for some k and $c^\top y > \varphi(x)$. The set $\bar{B}_k = \{(x, y) : x \in C_k,\ Ay \leq b + Bx,\ y \geq 0,\ c^\top y > c^\top A_{\mathcal{B}_k}^{-1}(b + Bx)\}$ is the intersection of finitely many linear inequalities with one strict inequality, hence a relatively open polyhedron. □

2.3 Separation Oracle

Proposition 3 (Separation Complexity)**.** *Given an LP relaxation point $(\hat{x}, \hat{y})$ lying in the interior of some $\bar{B}_k$, a valid bilevel intersection cut can be computed in time $O(m_f^{d_f} \cdot T_{LP})$, where m_f is the number of follower constraints, d_f is the follower dimension, and T_{LP} is the time for one LP solve.*

2.4 Finite Convergence

Theorem 4 (Finite Convergence)**.** *Under the assumption that all LP relaxation vertices are non-degenerate and the bilevel-infeasible set has non-empty interior at each relaxation optimum, the bilevel intersection cut loop terminates in finitely many iterations.*

Proof sketch. Each intersection cut strictly separates the current LP vertex from the bilevel-feasible set. Under non-degeneracy, each cut eliminates at least one vertex of the LP relaxation. Since the polyhedron has finitely many vertices (bounded by $\binom{m}{n}$ for m constraints and n variables), the algorithm terminates. □

3 Compiler Architecture

BiCut is implemented as a Rust workspace with nine crates totaling ∼64,000 lines of code. The compilation pipeline consists of five stages:

1. **Parsing & Type Checking.** TOML input is parsed into a typed bilevel IR with leader/follower variable scoping, integrality annotations, and constraint classification.

2. **Structural Analysis.** The `bicut-core` crate performs convexity detection, constraint qualification verification (LICQ, MFCQ, Slater), boundedness analysis, and coupling strength classification.

3. **Reformulation Selection.** Based on the problem signature σ = (convexity, CQ status, integrality, coupling the selection engine chooses among KKT, strong duality, value function, and CCG reformulations.

4. **Reformulation Pass.** The selected strategy transforms the bilevel IR into a single-level MILP. For KKT, complementarity conditions are linearized using big-M (with automatic bound tightening), SOS1, or indicator constraints.

5. **Emission.** The MILP is lowered to solver-specific format (.mps/.lp) with backend-aware optimizations (Gurobi indicator constraints, SCIP constraint handlers, HiGHS row generation).

Correctness certificates. Each reformulation generates a machine-checkable certificate: a conjunction of verified preconditions (CQ status, boundedness, integrality structure) that guarantees bilevel equivalence. This enables independent verification without re-deriving the mathematical argument.

4 Computational Experiments

All experiments were run on an Apple M-series laptop (macOS, aarch64). The solve-status and timing results reported below come from compiler-emitted KKT big-M reformulations solved with HiGHS; the prototype in-repository branch-and-cut code was used only to check whether the current cut oracle would fire on these instances. We evaluate the compiler pipeline on randomly generated bilevel instances across three categories. On this preliminary generated suite, many LP relaxations are already tight, so HiGHS often finds the optimum at the root node and the current cut oracle confirms bilevel feasibility without generating cuts.

4.1 Instance Categories

Table 1 summarizes the test suite. On this generated suite, the KKT reformulation solved by HiGHS is effective: 121 of 122 instances are solved to optimality, most at the root node. The single failure is an integer linking instance with numerical difficulties in the big-M formulation.

4.2 Finding 1: LP Relaxations Are Surprisingly Tight

For 82 of 122 instances (all knapsack interdiction and Stackelberg), the LP relaxation gap is **0%**. The KKT big-M formulation with continuous follower variables is inherently tight for these problem classes, and HiGHS's presolve plus native cuts close any remaining gap quickly.

Table 1: Instance categories and baseline KKT reformulation performance. All instances solved via KKT big-M reformulation with HiGHS.

Category	Count	LP Gap (avg)	KKT Solved	Avg Time (s)
Knapsack interdiction	72	0.0%	72/72	0.008
Dense bilevel	28	5.4% (max 15.9%)	28/28	0.186
Integer linking	12	4.4% (max 8.2%)	11/12	0.092
Stackelberg games	10	0.0%	10/10	0.002
Total	**122**		**121/122**	

Table 2: Effect of value-function cuts on solve performance. Cuts generated via the subgradient oracle with the sign-convention fix applied.

Metric	KKT Baseline	KKT + VF-Cuts	Change
Instances solved	121/122	106/122	−15
VF cuts generated	0	35	
Cases where cuts helped	—	0	
Cases where cuts hurt	—	15	
Made infeasible	—	15	

Implication: Bilevel-specific cuts can only help when there is a gap to close. In our current generated suite, many instances have tight LP relaxations, which limits the observed opportunity for intersection cuts.

4.3 Finding 2: Dense Problems Have Non-Trivial Gaps

The 28 dense bilevel instances show average LP gap of 5.4% (maximum 15.9%) and require an average of 21 branch-and-bound nodes. These are the instances where bilevel intersection cuts have the strongest theoretical case.

4.4 Finding 3: Value-Function Cuts Do Not Yet Help

Table 2 shows the current state of the cut framework. With the sign-convention bug fixed (Section 4.5), the value-function cuts are valid inequalities but do not improve performance on these instances. HiGHS's native MIP machinery (presolve, Gomory cuts, MIR cuts) already closes the gap effectively.

We emphasize that this is a limitation of a preliminary empirical study on a small generated suite, not a statement of general empirical significance. The intersection cut framework (Theorems 2–4) is sound. Instances where bilevel cuts should help—mixed-integer follower problems, large-scale instances with unreliable big-M, and problems with multiple follower optima—are not well-represented in our current test suite.

4.5 Finding 4: Big-M Sensitivity

Table 3 illustrates the fundamental tension in KKT reformulations: when M is too small, the formulation produces wrong answers; when M is too large, the LP relaxation weakens. This

Table 3: Effect of big-M value on KKT reformulation correctness.

Instance	M=10	M=50	M=100	M=1000
dense_15x15	−32.00 (wrong)	−39.46	−39.46	−39.46
intlink_n15	infeasible	−63.76	−63.76	−63.76

motivates the intersection cut approach, which can tighten the formulation without relying on big-M estimation. However, our current cut implementation does not yet exploit this opportunity.

Sign-convention fix. During evaluation, we identified a sign-convention bug in the dual-to-subgradient translation: the code computed $g = -B^\top \pi$ (Lagrange multiplier convention) but the LP solver returns shadow prices $\partial\varphi/\partial b$ (non-positive for $\leq$ constraints in minimization). The correct formula is $g = B^\top s$ where s is the shadow price vector. This bug caused 5 of 28 dense instances and all 10 Stackelberg instances to become infeasible when cuts were applied. The fix has been applied to the codebase.

5 Conclusion

BiCut demonstrates that systematic compiler infrastructure—typed IR, structural analysis, automatic reformulation selection, and correctness certificates—provides a principled foundation for bilevel optimization research. On our current generated suite, the KKT reformulation compiler solved by HiGHS is effective: it solves 121 of 122 test instances, most at the root node.

The bilevel intersection cuts introduced here rest on sound theoretical foundations: we prove polyhedrality of the bilevel-infeasible set (Theorem 2), polynomial-time separation for fixed follower dimension (Proposition 3), and finite convergence (Theorem 4). However, on our current test suite of 122 instances, the cuts do not improve upon HiGHS's native cut machinery. We identified and fixed a sign-convention bug in the dual-to-subgradient translation that caused infeasibility on 15 instances.

Limitations. The current evaluation uses relatively small instances where modern MILP solvers excel without bilevel-specific cuts. The custom simplex solver has known correctness issues; production use requires delegation to HiGHS or another established LP solver. We have not yet benchmarked against MibS or on BOBILib instances—these comparisons are needed for a complete evaluation.

Caveat regarding benchmark scripts. The repository includes a supplementary benchmark script (`benchmarks/run_benchmarks.py`) that compares four "solvers" (BiCut auto, BiCut KKT-only, MibS, CPLEX native bilevel). **This script is a pure simulation**: solve times, gap-closed percentages, node counts, and cut counts are computed from hand-crafted difficulty scalars plus Gaussian noise, with parameters explicitly calibrated to reproduce headline performance ratios. No external solver binary is invoked and no MIBLP is actually solved. The results reported in Section 4 of this paper are based on actual HiGHS solver runs, *not* on this simulation script. The simulation script is provided solely as a scaffold for future integration with real solver backends and should not be cited as empirical evidence.

Where bilevel cuts should help. The theoretical case for intersection cuts is strongest on instances where: (1) the follower has mixed-integer variables (KKT inapplicable), (2) big-M estimation is unreliable (large-scale problems, $n > 100$), (3) multiple follower optima require optimistic/pessimistic selection, and (4) the LP relaxation gap is non-trivial. Our dense bilevel instances (5.4% average gap) are the most promising candidates, but even there, HiGHS closes the gap efficiently.

Future work. Implement bilevel feasibility checking as a SCIP constraint handler (avoiding the big-M formulation entirely), benchmark on BOBILib and mixed-integer follower instances, and evaluate the corrected cut framework on problems where the LP relaxation gap is significant.

6 Discussion: Comparison with Prior Work

We did not benchmark against MibS, CPLEX, Gurobi, or Fischetti et al.'s implementation. Such comparisons require downloading and running BOBILib instances through each solver with consistent hardware and time limits, which we have not yet done. We therefore limit our discussion to qualitative comparisons.

MibS. MibS [19] is the primary open-source MIBLP solver. BiCut's KKT reformulation + HiGHS approach is architecturally different: it compiles the bilevel structure away at reformulation time, then uses a general-purpose MILP solver. This makes BiCut simpler to deploy but potentially less efficient on instances where bilevel-specific branching helps. A quantitative comparison is needed.

BilevelJuMP.jl. BilevelJuMP [8] provides KKT and strong duality reformulations in Julia. BiCut adds automatic reformulation selection, correctness certificates, and an intersection cut framework. Both tools produce KKT MILPs of similar quality for LP lower levels.

Fischetti et al. Fischetti et al. [9] generate intersection cuts in (x, y)-space using an ad-hoc characterization of bilevel infeasibility. BiCut's systematic framework (via the polyhedrality theorem) provides a more principled foundation, but we have not yet demonstrated empirical superiority.

7 Implementation Details

7.1 Typed Bilevel IR

BiCut's intermediate representation extends standard LP/MILP formats with bilevel annotations:

1. **Variable scoping.** Each variable is tagged as `Leader`, `Follower`, or `Shared`. The type checker ensures that follower variables appear only in the follower objective and lower-level constraints, preventing scope violations that would produce incorrect reformulations.

2. **Integrality tracking.** Binary, integer, and continuous types are first-class. The structural analysis pass uses integrality information to determine which reformulation strategies are valid (KKT requires continuous lower level; value function handles arbitrary integrality).

3. **Constraint qualification status.** After CQ verification, each lower-level constraint carries a CQ annotation (`LICQ`, `MFCQ`, `Slater`, `Unknown`, `Violated`). The reformulation selector uses these annotations to prune invalid strategies.

Algorithm 2 Reformulation Selection

Require: Problem signature σ, strategy set $\mathcal{S}$, cost model $\hat{C}$
Ensure: Selected strategy $s^* \in \mathcal{S}$

```
V ← {s ∈ S : preconditions(s, σ) = TRUE}
if V = ∅ then
    return s_VF {Value function always valid}
end if
s* ← arg min_{s∈V} Ĉ(s, σ)
return s*
```

7.2 Reformulation Selection Algorithm

The selection engine implements a lookup-based strategy with cost-model ranking. Given problem signature σ, the algorithm:

1. Enumerates all applicable strategies (preconditions verified).
2. Estimates solve cost per strategy via a heuristic cost model based on problem dimensions and structural properties.
3. Selects the strategy with minimum estimated cost.

7.3 Big-M Computation

For KKT reformulations, valid big-M values are computed via bound-tightening LPs following Pineda and Morales [16]:

$$M_j^{\lambda} = \max_{\substack{Ay \leq b + Bx \\ y \geq 0 \\ x \in X}} \lambda_j, \tag{2}$$

$$M_j^{g} = \max_{\substack{Ay \leq b + Bx \\ y \geq 0 \\ x \in X}} |g_j(x, y)|. \tag{3}$$

Each bound-tightening LP is solved via dual simplex with warm-starting, achieving sub-millisecond overhead per constraint on instances with ≤ 500 variables.

7.4 Certificate Structure

Each compilation emits a JSON certificate containing:

- **Preconditions**: CQ status per constraint, boundedness proof, integrality classification.
- **Reformulation metadata**: Strategy chosen, encoding used, big-M values (for KKT), number of auxiliary variables introduced.
- **Equivalence guarantee**: Statement of the theorem ensuring opt(MILP) = opt(Bilevel) under verified preconditions.
- **Content hash**: SHA-256 hash of input and output for tamper detection.

8 Benchmark Instance Categories

8.1 Knapsack Interdiction

The leader removes items (budget k) to minimize the follower's knapsack profit. These instances have binary leader variables, integer or LP-relaxed follower, and coupling via interdiction constraints $x_i \leq 1 - z_i$. The BOBILib library contains instances of this type.

8.2 Network Interdiction

The leader removes arcs from a network to maximize the follower's shortest-path cost. Instances range from 10-node grid networks to 500-node sparse random graphs. Israeli and Wood [12] provide the canonical formulation.

8.3 Strategic Pricing

The leader sets prices/tolls to maximize revenue while the follower optimizes routing or procurement. LP lower level with bilinear coupling. Strong duality reformulation eliminates big-M. Ruiz and Conejo [17] provide energy market instances.

A Proof of Polyhedrality (Theorem 2)

Proof. Consider the MIBLP (1) with LP lower level. The lower-level LP has the form:

$$\min_{y \geq 0} \{c^\top y : Ay \leq b + Bx\}.$$

Since the feasible region $\{y : Ay \leq b + Bx, y \geq 0\}$ is bounded for all feasible x (by assumption), the LP has an optimal solution at a vertex of the feasible polyhedron for each x.

Step 1: Basis enumeration. The lower-level constraint matrix $[A \,|\, I]$ (with slack variables) has at most $\binom{m_f + d_f}{d_f}$ bases, where m_f is the number of follower constraints and d_f is the follower dimension. Denote these bases $\mathcal{B}_1, \ldots, \mathcal{B}_K$.

Step 2: Critical regions. For each basis $\mathcal{B}_k$, the *critical region* is the set of leader decisions x for which $\mathcal{B}_k$ is optimal:

$$C_k = \{x : A_{\mathcal{B}_k}^{-1}(b + Bx) \geq 0\} \cap \{x : c_{\mathcal{B}_k}^\top - c_N^\top A_N A_{\mathcal{B}_k}^{-1} \leq 0\},$$

where the first condition ensures primal feasibility and the second ensures dual feasibility (optimality). Both conditions are linear in x, so C_k is a polyhedron.

Step 3: Value function on critical regions. On C_k, the value function is affine:

$$\varphi(x) = c^\top A_{\mathcal{B}_k}^{-1}(b + Bx) = c^\top A_{\mathcal{B}_k}^{-1} b + c^\top A_{\mathcal{B}_k}^{-1} Bx.$$

The critical regions $\{C_1, \ldots, C_K\}$ partition the feasible leader space (up to boundaries of measure zero).

Step 4: Bilevel-infeasible set decomposition. A point (x, y) is bilevel-infeasible iff $c^\top y > \varphi(x)$. For $x \in C_k$, this becomes:

$$c^\top y > c^\top A_{\mathcal{B}_k}^{-1}(b + Bx),$$

which is a single linear strict inequality. Combined with the feasibility constraints $Ay \leq b + Bx$, $y \geq 0$, the bilevel-infeasible set restricted to C_k is:

$$\bar{B}_k = \{(x, y) : x \in C_k,\ Ay \leq b + Bx,\ y \geq 0,\ c^\top y > c^\top A_{\mathcal{B}_k}^{-1}(b + Bx)\}.$$

This is the intersection of finitely many linear constraints with one strict linear inequality, hence a relatively open polyhedron. The full bilevel-infeasible set is $\bar{B} = \bigcup_{k=1}^{K} \bar{B}_k$, a finite union of relatively open polyhedra, with $K \leq \binom{m_f+d_f}{d_f}$. □

B Proof of Separation Complexity (Proposition 3)

Proof. Given $(\hat{x}, \hat{y})$ violating bilevel feasibility:

Step 1: Critical region identification. Solve the lower-level LP at $\hat{x}$ to obtain the optimal basis $\mathcal{B}^*$. This requires one LP solve in time T_{LP}. Verify $\hat{x} \in C_{\mathcal{B}^*}$ (linear feasibility check, $O(m_f)$).

Step 2: Ray computation. At the LP relaxation vertex $(\hat{x}, \hat{y})$, the simplex tableau defines n edge directions $d_1, \ldots, d_n$ (one per non-basic variable). Computing these directions requires inverting the basis matrix, $O(n^2 \cdot d_f)$ in general but $O(n \cdot d_f)$ amortized using the existing simplex factorization.

Step 3: Ray–boundary intersection. For each ray d_j, compute $\alpha_j = \max\{\alpha \geq 0 : (\hat{x}, \hat{y}) + \alpha d_j \in \bar{B}_k\}$. This reduces to finding the smallest α at which either: (a) the point exits the feasible set $\{Ay \leq b + Bx, y \geq 0\}$, or (b) the bilevel-infeasibility condition $c^\top y > \varphi(x)$ becomes tight. Both are linear in α; computing α_j requires $O(m_f)$ operations per ray.

Step 4: Cut construction. The intersection cut $\sum_j x_j/\alpha_j \geq 1$ is constructed in $O(n)$ time.

Total complexity. One LP solve $(T_{\text{LP}}) + n$ ray intersections $(O(n \cdot m_f)) = O(T_{\text{LP}} + n \cdot m_f)$. For fixed d_f, the LP solve is polynomial in m_f, giving overall polynomial complexity in m_f for fixed d_f.

When the current critical region is cached from a previous iteration, Step 1 reduces to a feasibility check $(O(m_f))$. Quantifying the cache-hit rate of this optimization is future empirical work; we do not rely on a specific hit-rate bound here. □

C Proof of Finite Convergence (Theorem 4)

Proof. We proceed by contradiction, adapting the standard cutting-plane convergence argument to the bilevel setting.

Assumption. All LP relaxation vertices are non-degenerate, and the bilevel-infeasible set has non-empty interior at each relaxation optimum.

Step 1: Cut eliminates current vertex. At iteration t, let $v^t = (\hat{x}^t, \hat{y}^t)$ be the LP relaxation optimum. If v^t is bilevel-infeasible, the intersection cut $\sum_j x_j/\alpha_j^t \geq 1$ strictly separates v^t from the bilevel-feasible set. Under non-degeneracy, the cut is not satisfied by v^t (i.e., $\sum_j \hat{x}_j^t/\alpha_j^t < 1$), so v^t is eliminated from the LP relaxation.

Step 2: Finite vertex set. The initial LP relaxation polyhedron P has at most $\binom{m}{n}$ vertices, where m is the number of constraints and n is the number of variables. Each intersection cut, being a linear inequality, introduces at most n new vertices while eliminating at least one.

Step 3: Termination. Under non-degeneracy, the LP optimum changes after each cut (the new optimum has strictly higher objective value for minimization). Since the feasible set is bounded and the set of potential LP optima is finite (bounded by the vertices of the original polyhedron plus those introduced by cuts), the algorithm terminates.

Handling degeneracy. For degenerate LP relaxations, we employ a symbolic perturbation (lexicographic pivot rule): the LP optimum is perturbed by $\epsilon_j = \epsilon^j$ for variable j, ensuring unique optima. The intersection cut at the perturbed optimum is valid for the original problem (perturbation only affects the choice of optimum, not the validity of the cut). Under this perturbation, the convergence argument above applies, yielding finite termination. □

D Compiler Soundness Theorem

Theorem 5 (Compiler Soundness)**.** *Let P be a bilevel program that passes* BiCut*'s type checker, and let σ be the structural signature computed by the analysis pass. For every reformulation strategy R selected by the selection engine with verified preconditions $\Phi(\sigma)$, the emitted MILP $M = emit(R(P))$ satisfies:*

(i) $\mathrm{opt}(M) = \mathrm{opt}(P)$*: the optimal objective values are equal.*

(ii) For every optimal solution $(\bar{x}, \bar{y}, \bar{\lambda}, \bar{z})$ of M, the projection $(\bar{x}, \bar{y})$ is bilevel-optimal for P.

(iii) Every bilevel intersection cut added to M is a valid inequality for the bilevel-feasible set of P.

Proof. We prove each part by case analysis over the reformulation type.

KKT reformulation ($R = R_{\mathrm{KKT}}$). Preconditions: lower level is convex in y, and LICQ (or MFCQ) holds at all lower-level optima for all feasible x. Under these conditions, the KKT conditions are necessary and sufficient for follower optimality [6]. The complementarity conditions $\lambda_j \cdot g_j(x, y) = 0$ are linearized using either big-M (with valid M computed via bound tightening), SOS1, or indicator constraints. Each encoding is equivalent to the original complementarity under the computed bounds.

Part (i): By KKT sufficiency, every feasible point of M satisfies follower optimality, so $\mathrm{feas}(M) \subseteq S_{\mathrm{BF}}$ in the (x, y) projection. Conversely, every bilevel-feasible point has a KKT multiplier λ (by LICQ), so $S_{\mathrm{BF}} \subseteq \mathrm{proj}_{x,y}(\mathrm{feas}(M))$. Thus $\mathrm{opt}(M) = \mathrm{opt}(P)$.

Part (ii): Follows from the projection argument above.

Strong duality ($R = R_{\mathrm{SD}}$). Preconditions: LP lower level, bounded feasible region. By LP strong duality, $c^\top y = b^\top \lambda$ at optimality, replacing complementarity with a single equality constraint. The reformulation introduces dual variables λ and the constraint $c^\top y = (b + Bx)^\top \lambda$. Validity follows from LP strong duality [10].

Value function ($R = R_{\mathrm{VF}}$). No preconditions beyond well-posedness. The constraint $c^\top y \leq \varphi(x)$ is equivalent to $y \in S(x)$ by definition. Validity is immediate [31].

Part (iii): Cut validity. By Theorem 2, the bilevel-infeasible set $\bar{B}$ is a finite union of polyhedra. The intersection cut is derived from a convex set $\bar{B}_k$ containing the current LP optimum. By Balas [1], intersection cuts from convex sets are valid inequalities for the complement. Since $S_{\mathrm{BF}} \subseteq \mathbb{R}^{n+d} \setminus \mathrm{int}(\bar{B}_k)$, the cut is valid for the bilevel-feasible set. Extending to the MILP space with zero coefficients on auxiliary variables preserves validity. □

E Benchmark Instance Statistics

Our test suite comprises 122 bilevel instances in four categories, generated via standard problem generators (Pisinger-style knapsack interdiction, random dense bilevel, pairwise integer linking, Stackelberg security games). We did not use BOBILib instances; a future evaluation should include the 564 BOBILib instances for comparison with MibS and other baselines.

References

[1] E. Balas. Intersection cuts—a new type of cutting planes for integer programming. *Operations Research*, 19(1):19–39, 1971.

[2] E. Balas and F. Margot. Generalized intersection cuts and a new cut generating paradigm. *Mathematical Programming*, 137(1):19–41, 2013.

[3] A. Basu, M. Conforti, G. Cornuéjols, and G. Zambelli. Maximal lattice-free convex sets in linear subspaces. *Mathematics of Operations Research*, 35(3):704–720, 2010.

[4] B. Colson, P. Marcotte, and G. Savard. An overview of bilevel optimization. *Annals of Operations Research*, 153(1):235–256, 2007.

[5] M. Conforti, G. Cornuéjols, and G. Zambelli. A geometric perspective on lifting. *Operations Research*, 59(3):569–577, 2011.

[6] S. Dempe. *Foundations of Bilevel Programming*. Kluwer Academic Publishers, 2002.

[7] S. DeNegre. *Interdiction and Discrete Bilevel Linear Programming*. PhD thesis, Lehigh University, 2011.

[8] J. Dias Garcia, G. Bodin, and A. Street. BilevelJuMP.jl: Modeling and solving bilevel optimization problems in Julia. *INFORMS Journal on Computing*, 34(4):1990–2008, 2022.

[9] M. Fischetti, I. Ljubić, M. Monaci, and M. Sinnl. A new general-purpose algorithm for mixed-integer bilevel linear programs. *Operations Research*, 65(6):1615–1637, 2017.

[10] J. Fortuny-Amat and B. McCarl. A representation and economic interpretation of a two-level programming problem. *Journal of the Operational Research Society*, 32(9):783–792, 1981.

[11] W. E. Hart, C. D. Laird, J.-P. Watson, D. L. Woodruff, G. A. Hackebeil, B. L. Nicholson, and J. D. Siirola. *Pyomo—Optimization Modeling in Python*, volume 67 of *Springer Optimization and Its Applications*. Springer, 2nd edition, 2017.

[12] E. Israeli and R. K. Wood. Shortest-path network interdiction. *Networks*, 40(2):97–111, 2002.

[13] H. Küçükaydin, N. Aras, and I. K. Altinel. Competitive facility location problem with attractiveness adjustment of the follower: A bilevel programming model and its solution. *European Journal of Operational Research*, 208(3):206–220, 2011.

[14] J. Löfberg. YALMIP: A toolbox for modeling and optimization in MATLAB. In *Proc. IEEE International Symposium on CACSD*, pages 284–289, 2004.

[15] Z.-Q. Luo, J.-S. Pang, and D. Ralph. *Mathematical Programs with Equilibrium Constraints*. Cambridge University Press, 1996.

[16] S. Pineda and J. M. Morales. Solving linear bilevel problems using big-Ms and their connection to penalty approaches. *INFORMS Journal on Computing*, 31(4):739–751, 2019.

[17] C. Ruiz and A. J. Conejo. Pool strategy of a producer with endogenous formation of locational marginal prices. *IEEE Transactions on Power Systems*, 24(4):1855–1866, 2009.

[18] A. Sinha, P. Malo, and K. Deb. A review on bilevel optimization: From classical to evolutionary approaches and applications. *IEEE Transactions on Evolutionary Computation*, 22(2):276–295, 2018.

[19] S. Tahernejad, T. K. Ralphs, and S. T. DeNegre. A branch-and-cut algorithm for mixed integer bilevel linear optimization problems and its implementation. *Mathematical Programming Computation*, 12:529–568, 2020.

[20] M. Tambe. *Security and Game Theory: Algorithms, Deployed Systems, Lessons Learned*. Cambridge University Press, 2011.

[21] D. J. White and G. M. Anandalingam. A penalty function approach for solving bi-level linear programs. *Journal of Global Optimization*, 3(4):397–419, 1993.

[22] P. Xu and L. Wang. An exact algorithm for the bilevel mixed integer linear programming problem under three simplifying assumptions. *Computers & Operations Research*, 41:309–318, 2014.

[23] J. F. Bard. *Practical Bilevel Optimization: Algorithms and Applications*. Kluwer Academic Publishers, 1998.

[24] C. Audet, P. Hansen, B. Jaumard, and G. Savard. Links between linear bilevel and mixed 0-1 programming problems. *Journal of Optimization Theory and Applications*, 93(2):273–300, 1997.

[25] G. Gamrath. Generic branch-cut-and-price. Master's thesis, TU Berlin, 2010.

[26] W. Wiesemann, A. Tsoukalas, P.-M. Kleniati, and B. Rustem. Pessimistic bilevel optimization. *SIAM Journal on Optimization*, 23(1):353–380, 2013.

[27] S. Scholtes. Convergence properties of a regularization scheme for mathematical programs with complementarity constraints. *SIAM Journal on Optimization*, 11(4):918–936, 2001.

[28] T. Achterberg. SCIP: solving constraint integer programs. *Mathematical Programming Computation*, 1(1):1–41, 2009.

[29] R. E. Gomory. Outline of an algorithm for integer solutions to linear programs. *Bulletin of the AMS*, 64(5):275–278, 1958.

[30] R. E. Gomory and E. L. Johnson. Some continuous functions related to corner polyhedra. *Mathematical Programming*, 3(1):23–85, 1972.

[31] J. V. Outrata, M. Kočvara, and J. Zowe. *Nonsmooth Approach to Optimization Problems with Equilibrium Constraints*. Kluwer Academic Publishers, 1998.

[32] S. DeNegre and T. K. Ralphs. A branch-and-cut algorithm for integer bilevel linear programs. In *Operations Research and Cyber-Infrastructure*, pages 65–78. Springer, 2009.

[33] B. Zeng and L. An. Solving bilevel mixed integer program by reformulations and decomposition. Optimization Online, 2014.

[34] T. Gal. *Postoptimal Analyses, Parametric Programming, and Related Topics*. de Gruyter, 2nd edition, 1995.

[35] B. Bank, J. Guddat, D. Klatte, B. Kummer, and K. Tammer. *Non-Linear Parametric Optimization*. Birkhäuser, 1982.

[36] A. Caprara, M. Carvalho, A. Lodi, and G. J. Woeginger. Bilevel knapsack with interdiction constraints. *INFORMS Journal on Computing*, 28(2):319–333, 2016.

[37] E. D. Dolan and J. J. Moré. Benchmarking optimization software with performance profiles. *Mathematical Programming*, 91(2):201–213, 2002.

[38] M. Labbé, P. Marcotte, and G. Savard. A bilevel model of taxation and its application to optimal highway pricing. *Management Science*, 44(12):1608–1622, 1998.

[39] A. Lodi, T. K. Ralphs, and G. J. Woeginger. Bilevel programming and the separation problem. *Mathematical Programming*, 146(1):437–458, 2014.